

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

```
[2]: iris = load_iris()

X = iris.data
y = iris.target

print("Feature Shape:", X.shape)
print("Target Shape:", y.shape)
```

```
Feature Shape: (150, 4)
Target Shape: (150,)
```

```
[3]: df = pd.DataFrame(X, columns=iris.feature_names)
df["Species"] = y

df.head()
```

```
[3]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
[4]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype  
---  -
 0   sepal length (cm)  150 non-null   float64
 1   sepal width (cm)   150 non-null   float64
 2   petal length (cm)  150 non-null   float64
 3   petal width (cm)   150 non-null   float64
 4   Species            150 non-null   int64   
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
[5]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (120, 4)
Testing Data: (30, 4)
```

```
[6]: scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
[7]: svm = SVC(kernel="linear")
```

```
svm.fit(X_train, y_train)
```

```
svm_pred = svm.predict(X_test)
```

```
print("SVM Accuracy:", accuracy_score(y_test, svm_pred))
```

```
SVM Accuracy: 0.9666666666666667
```

```
[8]: print(classification_report(y_test, svm_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

```
[9]: cm = confusion_matrix(y_test, svm_pred)
```

```
plt.figure(figsize=(6,5))
```

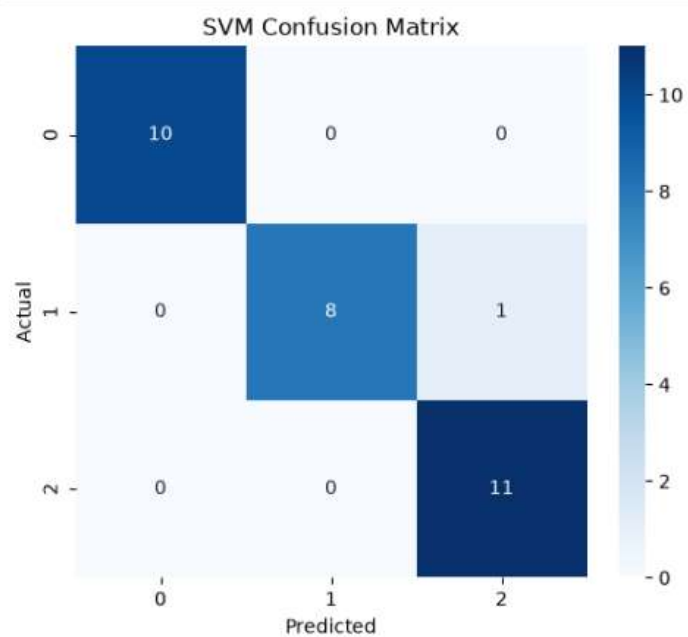
```
sns.heatmap(  
    cm,  
    annot=True,  
    cmap="Blues",  
    fmt="d"  
)
```

```
plt.title("SVM Confusion Matrix")
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.show()
```



```
[10]: rf = RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )

    rf.fit(X_train, y_train)

    rf_pred = rf.predict(X_test)

    print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))

Random Forest Accuracy: 1.0
```

```
[11]: print(classification_report(y_test, rf_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

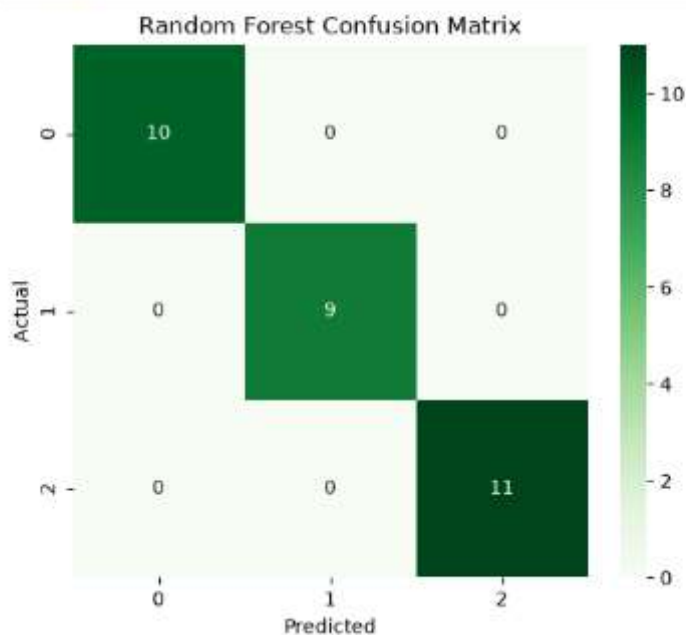
```
[12]: cm = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    cmap="Greens",
    fmt="d"
)

plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



FIGURE

```
[13]: svm_accuracy = accuracy_score(y_test, svm_pred)
      rf_accuracy = accuracy_score(y_test, rf_pred)

      print("SVM Accuracy :", svm_accuracy)
      print("Random Forest Accuracy :", rf_accuracy)

      SVM Accuracy : 0.9666666666666667
      Random Forest Accuracy : 1.0
```

[]:

